FC7xxx LIN User Manual

Rev.0.6

Revision History

Revision	Date	Changes
0.1	2023/07/14	Initial release for MCAL V0.1.0
0.3	2023/10/20	Updated for MCAL V0.3.0
0.6	2023/03/29	Updated for MCAL V0.6.0 - Added support for FC7240

Table of Contents

Chapter 1 LIN Introduction

1.1 Requirements

The design of this module follows the specifications of the Lin driver specified in AUTOSAR Classic Platform Release 4.6.0. For detailed requirements, refer to the AUTOSAR_SWS_LINDriver.

1.2 Design Summary

The LIN driver is a part of the microcontroller abstraction layer (MCAL), performing the hardware access and offers a hardware independent API to the upper layer. The only upper layer, which can access to the LIN driver, is the LIN Interface. The LIN driver can support more than one channel. This means that the LIN driver can handle one or more LIN channels as long as they belong to the same LIN hardware unit.

The LIN Driver for FC7xxx uses FCUART on-chip hardware modules which provides special support for the LIN protocol. The hardware modules can be used to automate most tasks of a LIN master. The LIN physical interface should be connected to the FCUART module pins in order to get the LIN bus voltage levels.

1.3 Hardware Summary

Features of the FC Universal Asynchronous Receiver Transmitter (FCUART) module include:

- Full-duplex, standard non-return-to-zero (NRZ) format
- Independent FIFO structure for transmitting and receiving:
  - Both Transmit and Receive data FIFO depth are
  - Watermark for receiving and transmitting requests are separate configurable
  - Option for the receiver to assert request after a configurable number of idle characters if receive FIFO is not empty Programmable baud rate and oversampling ratio
- Use receiver monitor other FCUART transmitter
- Programmable data length from 7-bit to 10-bit
- Configurable transmit output and receive input polarity
- Multiple interrupt or DMA request source
- Transmitter and receiver buffer data parity error check
- Programmable 1-bit or 2-bit stop bits
- Support operation in Stop mode; FCUART0 supports operation in standby mode
- Three receiver wakeup methods including address mark wakeup, idle line wakeup and receive data match
- Automatic address matching to reduce ISR overhead
  - Address mark matching
  - Idle line address matching
  - Address match start, address match end
- Optional 13-bit break character generation / 11-bit break character detection
- Configurable idle length detection supporting 1, 2, 4, 8, 16, 32, 64 or 128 idle characters
- Request to send (RTS) and clear to send (CTS) hardware control flow support
- The FCUART supports full-duplex, asynchronous, NRZ serial communication and comprises a baud rate generator, transmitter, and receiver block. The transmitter and receiver use the same baud rate generate but can operate independently.

Chapter 2 Software Design

2.1 Rejected Requirements

Rejected Requirement 1 SWS_Lin_00026	
Description	If the LIN hardware unit cannot queue the bytes for transmission or reception (e.g. simple UART implementation), the LIN driver shall provide a temporary communication buffer.
Rejection Reason	FC7xxx uses FCUART to implement LIN and do not provide a temporary communication buffer.

Rejected Requirement 2 SWS_Lin_00039	
Description	Values that can be configured are hardware dependent. Therefore, the rules and constraints cannot be given in the standard.
Rejection Reason	Useless SWS in FC7xxx.

Rejected Requirement 3 SWS_Lin_00177	
Description	"In case several LIN driver instances (of same or different vendor) are implemented in one ECU the file names, API names, and published parameters must be modified such that no two definitions with the same name are generated. The name shall be extended according to SRS_BSW_00347 with a Vendor Id (needed to distinguish LIN drivers from different vendors) and a Vendor specific name (needed to distinguish different hardware units implemented by one Vendor): <Module abbreviation>_<Vendor Id>_<Vendor specific name>."
Rejection Reason	FC7xxx just has FCUART to implement LIN and do not support different Vendor mode

Rejected Requirement 4 SWS_Lin_00201	
Description	For different LIN hardware units, a separate LIN driver needs to be implemented. It is up to the implementer to adapt the driver to the different instances of similar LIN channels.
Rejection Reason	FC7xxx just has FCUART to implement LIN.

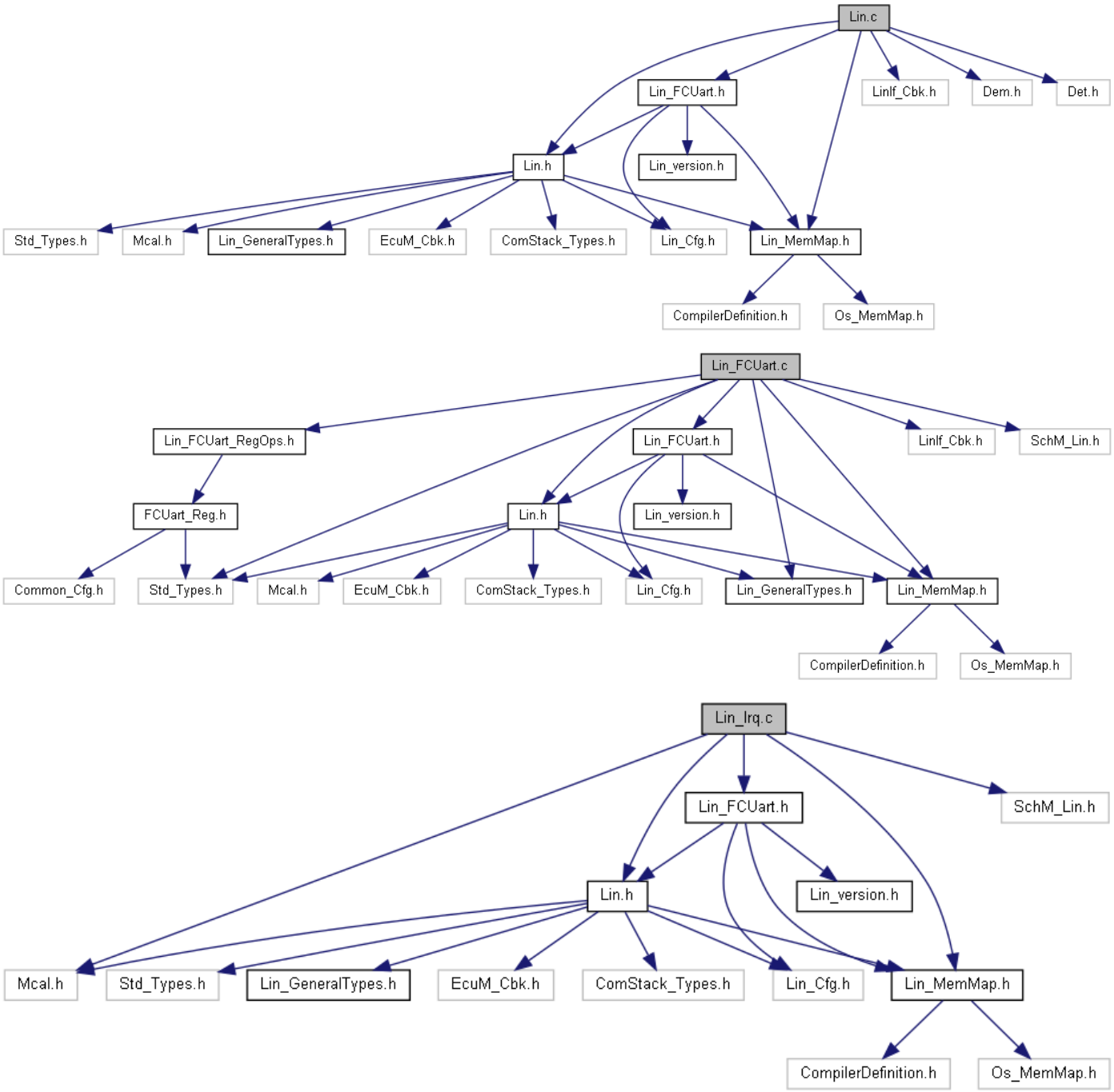
Rejected Requirement 5 SWS_Lin_00221	
Description	The function Lin_GoToSleep shall optionally set the LIN hardware unit to reduced power operation mode (if supported by HW), even in case of an erroneous transmission of the go-to-sleep-command.
Rejection Reason	FCUARTs have no reducded power operation mode.

Rejected Requirement 5 SWS_Lin_00245	
Description	The content of Lin_GeneralTypes.h shall be protected by a LIN_GENERAL_TYPES define.
Rejection Reason	The content of Lin_GeneralTypes.h shall be protected by a LIN_GENERAL_TYPES_H define.

Rejected Requirement 5 SWS_Lin_CONSTR_00279	
Description	LinChannel and LinTrcvChannel of one communication channel shall all reference the same ECUC partition.
Rejection Reason	LinTrcv module has not been implemented.

Rejected Requirement 5 SWS_Lin_CONSTR_00246	
Description	If different LIN drivers are used, only one instance of this file has to be included in the source tree. For implementation all Lin_GeneralTypes.h related types in the documents mentioned before shall be considered.
Rejection Reason	Only Uart was used to implement the Lin module.

2.2 File Structure



2.3 Macros

2.3.1 Macros in Lin.h

- #define **LIN_INSTANCE_ID** ((uint8)0U)

  LIN Instance ID.

- #define **LIN_E_UNINIT** ((uint8)0x00U)

  API service used without ADC module initialization.

- #define **LIN_E_INVALID_CHANNEL** ((uint8)0x02U)

API service used with an invalid or inactive channel parameter.

- #define **LIN_E_INVALID_POINTER** ((uint8)0x03U)
  API service called with invalid configuration pointer.

- #define **LIN_E_STATE_TRANSITION** ((uint8)0x04U)
  Invalid state transition for the current state.

- #define **LIN_E_PARAM_POINTER** ((uint8)0x05U)
  API service called with a NULL pointer.

- #define **LIN_E_TIMEOUT** ((uint8)0x06U)
  Timeout caused by hardware error.

- #define **LIN_E_ALREADY_INITIALIZED** ((uint8)0x07U)
  Lin Already Initialized.
- #define **LIN_INIT_ID** ((uint8)0x0U)
  API Service ID for Lin_Init().

- #define **LIN_VERSIONINFO_ID** ((uint8)0x1U)
  API Service ID for Lin_GetVersionInfo().

- #define **LIN_SENDFRAME_ID** ((uint8)0x4U)
  API Service ID for Lin_SendFrame().

- #define **LIN_GOTOSLEEP_ID** ((uint8)0x6U)
  API Service ID for Lin_GoToSleep().

- #define **LIN_WAKEUP_ID** ((uint8)0x7U)
  API Service ID for Lin_WakeUp().

- #define **LIN_GETSTATUS_ID** ((uint8)0x8U)
  API Service ID for Lin_GetStatus().

- #define **LIN_GOTOSLEEPINTERNAL_ID** ((uint8)0x9U)
  API Service ID for Lin_GoToSleepInternal().

- #define **LIN_CHECKWAKEUP_ID** ((uint8)0xAU)
  API Service ID for Lin_CheckWakeup().

- #define **LIN_WAKEUPINTERNAL_ID** ((uint8)0xBU)
  API Service ID for Lin_WakeUpInternal().

- #define **LIN_UNINIT** ((uint8)0x01U)
  Lin driver initialization states.

- #define **LIN_INIT** ((uint8)0x02U)
  Lin driver initialization states.

- #define **LIN_CH_SLEEP_PENDING** ((uint8)0x01U)
  Lin Channel states. go-to-sleep-command has been issued on the bus, Lin channel stay at this state until Lin_GetStatus() is called.

- #define **LIN_CH_SLEEP_STATE** ((uint8)0x02U)
  Lin Channel states. The detection of a wake-up pulse is enabled. The Lin hardware is into a low power mode if such a mode is provided by the hardware.
- #define **LIN_CH_OPERATIONAL_STATE** ((uint8)0x03U)
  Lin Channel states. The individual channel has been initialized (using at least one channel configured data set) and is able to participate in the Lin cluster.

- #define **LIN_CH_NOT_READY_STATE** ((uint8)0x04U)
  Lin Channel states. The individual channel is not ready to process a frame.

- #define **LIN_CH_READY_STATE** ((uint8)0x05U)
  Lin Channel states. The individual channel is ready to process a frame.

- #define **LIN_TX_COMPLETE_STATE** ((uint8)0x06U)
  Lin Channel states. Lin frame was sent and no errors.

- #define **LIN_RX_COMPLETE_STATE** ((uint8)0x07U)
  Lin Channel states. Lin frame was received and no errors.

- #define **LIN_CH_RECEIVE_NOTHING_STATE** ((uint8)0x08U)
  Lin Channel states. State after the Lin frame header was correctly sent.

- #define **LIN_RX_ONGOING_STATE** ((uint8)0x09U)
  Lin Channel states. Lin frame is receiving.

- #define **LIN_TX_HEADER_COMPLETE_STATE** ((uint8)0x10U)
  Lin Channel states. Lin header is transmission.

- #define **LIN_NO_ERROR** ((uint8)0x00U)
  Interrupt Errors conditions. No error occurred on a channel.

- #define **LIN_BIT_ERROR** ((uint8)0x01U)
  Interrupt Errors conditions. Bit error on a channel:
  - During response field transmission (Slave and Master modes);
  - During header transmission (in Master mode).

- #define **LIN_CHECKSUM_ERROR** ((uint8)0x02U)
  Interrupt Errors conditions. Checksum error on a channel.

- #define **LIN_SYNCH_FIELD_ERROR** ((uint8)0x03U)
  Interrupt Errors conditions. Inconsistent Synch Field.

- #define **LIN_BREAK_DELIMITER_ERROR** ((uint8)0x04U)
- Interrupt Errors conditions. Break Delimiter too short (< 1 bit).

- #define **LIN_IDENTIFIER_PARITY_ERROR** ((uint8)0x05U)
  Interrupt Errors conditions. Parity error.

- #define **LIN_FRAMING_ERROR** ((uint8)0x06U)
  Interrupt Errors conditions. Invalid stop bit:- During reception of any data in the response field (Slave and Master modes); - During reception of Synch or Identifier Field (Slave mode).

- #define **LIN_BUFFER_OVER_RUN_ERROR** ((uint8)0x07U)
  Interrupt Errors conditions. New data byte is received on a channel and the buffer full flag is not cleared.

- #define **LIN_NOISE_ERROR** ((uint8)0x08U)
  Interrupt Errors conditions. Noise detected on a received character.

- #define **LIN_TIMEOUT_ERROR** ((uint8)0x09U)
  Interrupt Errors conditions. Header or Response timeout detected.

- #define **LIN_BUFFER_UNDER_RUN_ERROR** ((uint8)0x0AU)
  Interrupt Errors conditions. Shifter was ready to load new data from RECVBUF before new data had been written into RECVBUF.

- #define **LIN_TX_MASTER_RES_COMMAND** ((uint8)0x01U)
  Commands IDs. Tx frame is a master frame (response is provided by master).

- #define **LIN_TX_SLAVE_RES_COMMAND** ((uint8)0x02U)
  Commands IDs. Tx frame is a slave frame (response is provided by slave).

- #define **LIN_TX_SLEEP_COMMAND** ((uint8)0x03U)
  Commands IDs. Tx frame is a sleep command frame.

- #define **LIN_TX_NO_COMMAND** ((uint8)0x04U)
  Commands IDs. No tx master command pending.

- #define **LIN_TX_SLAVE_TO_SLAVE_COMMAND** ((uint8)0x05U)
  Commands IDs. Tx frame is a slave frame.

2.3.2 Macros in Lin_version.h

- #define **LIN_AR_RELEASE_MAJOR_VERSION** 4
- #define **LIN_AR_RELEASE_MINOR_VERSION** 6
- #define **LIN_AR_RELEASE_REVISION_VERSION** 0
- #define **LIN_SW_MAJOR_VERSION** 0
- #define **LIN_SW_MINOR_VERSION** 6
- #define **LIN_SW_PATCH_VERSION** 0
- #define **LIN_VENDOR_ID** 174
- #define **LIN_MODULE_ID** 82

2.3.3 Macros in Lin_Cfg.h

- #define **LIN_CFG_VENDOR_ID** 174
- #define **LIN_CFG_MODULE_ID** 82
- #define **LIN_CFG_AR_RELEASE_MAJOR_VERSION** 4
- #define **LIN_CFG_AR_RELEASE_MINOR_VERSION** 6
- #define **LIN_CFG_AR_RELEASE_REVISION_VERSION** 0

- #define **LIN_CFG_SW_MAJOR_VERSION** 0

- #define **LIN_CFG_SW_MINOR_VERSION** 6

- #define **LIN_CFG_SW_PATCH_VERSION** 0

- #define **LIN_INSTANCE_COUNT** 18U
  Total number of available hardware FCUART channels. This define was enable when the platform has only one interrupt for each LINFlex channel.

- #define **LIN_INSTANCE_CONFIG** 4U
  Number of Channels configured. This define was enable when the platform has only one interrupt for each LINFlex channel.

- #define **LIN_FCUART_MAX_MODULES** 18U
  Total number of available hardware lin channels. This define was enable when the platform has only one interrupt for each LINFlex channel.

- #define **LIN_MAX_DATA_LENGTH** 8U
  Max data length of the LIN SDU buffer to be returned. This define was enable when the platform has only one interrupt for each LINFlex channel.

- #define **LIN_TIMEOUT_TIMES** ((uint32)1000U)
  Number of loops before returning LIN_E_TIMEOUT. This define was enable when the platform has only one interrupt for each LINFlex channel.

- #define **LIN_DEV_ERROR_DETECT** (STD_OFF)
  Switches the Development Error Detection and Notification ON or OFF. This define was enable when the platform has only one interrupt for each LINFlex channel.

- #define **LIN_VERSION_INFO_API** (STD_OFF)
  Support for version info API. Switches the Lin_GetVersionInfo() API ON or OFF.

- #define **LIN_PARTIONS_NB** ((uint32)1U)
  The number of partions of the Lin driver

- #define **LIN_DRIVER_STATUS_UNINIT_ARRAY** {LIN_UNINIT}
  Init variable driver status

- #define **LIN_CONFIGPTR_UNINIT_ARRAY** {NULL_PTR}
  Init variable Lin_ConfigPtr

- #define **LIN_CORE_CONFIGPTR_UINIT_ARRAY** {NULL_PTR, NULL_PTR, NULL_PTR, NULL_PTR}
  Init variable Lin_FCUart_pConfig

- #define **LIN_CHMAP_UNINT_ARRAY** {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}
  Init Lin channel hardware map variable

- #define **LIN_FCUART_UNALLOCATEDPAR_CORE_ID** ((uint32)0U)
  The number of Core Id

- #define **LIN_FRAME_TIMEOUT_DISABLE** (STD_OFF)
  Disable Frame Timeout Feature. When NonAutosar/LinFrameTimeoutDisable == STD_ON, the LIN Master will accept the frame that is longer than TxFrame_Maximum.

- #define **LIN_MULTICORE_SUPPORT** (STD_OFF)
  Multicore is enabled or not

- #define **FCUART_6** 6U
- Link Lin channels symbolic names with Lin channel IDs.

- #define **LIN_FCUART_6_ISR_USED** (STD_ON)
- FCUART Instance of ISR. define FCUART Instance of ISR

- #define **LIN_DISABLE_DEM_REPORT_ERROR_STATUS** (STD_ON)
  Switches the Production Error Detection and Notification OFF. Symbolic names for configured channels.

- #define **LIN_PRECOMPILE_SUPPORT** (STD_OFF)
  Symbolic names for configured channels.

- #define **LIN_NONE_ECUM_WAKEUP_SOURCE_REF** (uint32)0U
  None EcuMWakeUpSource was referred when LinChannelWakeupSupport is disable

2.4 Enums

2.4.1 Enums in Lin_GeneralTypes.h

2.4.1.1 Lin_FrameCsModelType

Enumeration	Lin_FrameCsModelType	
Description	Checksum models for the LIN Frame.	
Values	**Value**	**Description**
	LIN_ENHANCED_CS = 0U	Enhanced checksum model.
	LIN_CLASSIC_CS = 1U	Classic checksum model.

2.4.1.2 Lin_FrameResponseType

Enumeration	Lin_FrameResponseType	
Description	Frame response types.	
Values	**Value**	**Description**
	LIN_FRAMERESPONSE_TX = 0U	Response is generated from this (master) node.
	LIN_FRAMERESPONSE_RX = 1U	Response is generated from a remote slave node.
	LIN_FRAMERESPONSE_IGNORE = 2U	Response is generated from another node and is irrelevant for this node.

2.4.1.3 Lin_StatusType

Enumeration	Lin_StatusType	
Description	LIN Frame and Channel states operation.	
Values	**Value**	**Description**
	LIN_NOT_OK = 0U	Development or production error occurred
	LIN_TX_OK = 1U	Successful transmission.
	LIN_TX_BUSY= 2U	Ongoing transmission (Header or Response).
	LIN_TX_HEADER_ERROR= 3U	Erroneous header transmission such as: - Mismatch between sent and read back data - Identifier parity error - Physical bus error.
	LIN_TX_ERROR= 4U	Erroneous transmission such as: - Mismatch between sent and read back data - Physical bus error.
	LIN_RX_OK= 5U	Reception of correct response.
	LIN_RX_BUSY= 6U	Ongoing reception: at least one response byte has been received, but the checksum byte has not been received.
	LIN_RX_ERROR= 7U	Erroneous reception such as: - Framing error - Overrun error - Checksum error - Short response.
	LIN_RX_NO_RESPONSE= 8U	No response byte has been received so far.
	LIN_OPERATIONAL= 9U	Normal operation: - The related LIN channel is ready to transmit next header. - No data from previous frame available (e.g. after initialization).
	LIN_CH_SLEEP= 10U	Sleep mode operation; - In this mode wake-up detection from slave nodes is enabled.

2.4.2 Enums in Lin_Cfg.h

2.4.2.1 Lin_BreakDelimiterType

Enumeration	Lin_BreakDelimiterType	
Description	This is the type of the break delimiter for the Lin module.	
Values	**Value**	**Description**
	LEN_DELIMITER_1BIT = 0U	1-bit length for LIN break send out.
	LEN_DELIMITER_2BITS = 1U	2-bit length for LIN break send out.

2.5 Typedefs

2.5.1 Typedefs in Lin_GeneralTypes.h

- typedef uint8 **Lin_FrameDlType**

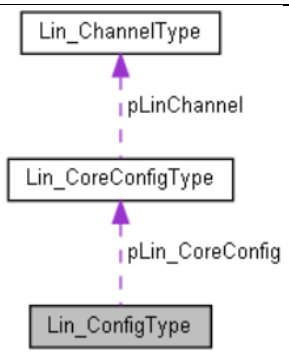
  Data length of a LIN Frame.

- typedef uint8 **Lin_FramePidType**

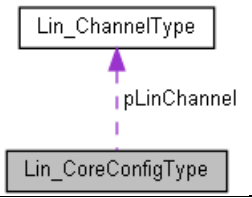
  The LIN identifier (0..0x3F) with its parity bits(bit6,bit7).

2.6 Structures

2.6.1 Lin_ConfigType

Structure	Lin_ConfigType
Description	LIN driver configuration type structure.
Diagram	
Data Fields	• const Lin_CoreConfigType *pLin_CoreConfig[FCUART_INSTANCE_MAX] • Pointer to core configuration.

2.6.2 Lin_CoreConfigType

Structure	Lin_CoreConfigType
Description	LIN core configuration type structure.
Diagram	
Data Fields	• const Lin_ChannelType *pLinChannel Pointer to Lin channel configure

	• Lin_BreakDelimiterType eLinBreakDelimiterLen; Lin break delimiter length. • uint32 u32LinBaudRate Lin baudrate configure. • uint32 u32LinSampSbr_RegVal Lin value of register SAMP multiply SBR. • uint16 u16LinBaudWaitCount Lin wait count of change baudrate.

2.6.3 Lin_ChannelType

Structure	Lin_ChannelType
Description	LIN channel configuration type structure.
Diagram	N/A
Data Fields	• uint8 u8LinChannelID • Lin Channel ID. • uint8 u8HwModule • Lin HW channel. • uint8 u8LinWakeUpSup • Identifies the Lin channel support wake up. • Uint32 LinChannelWakeUpSrc • Identifies the wake up source • uint32 ChannelCoreId • Lin Channel Core ID

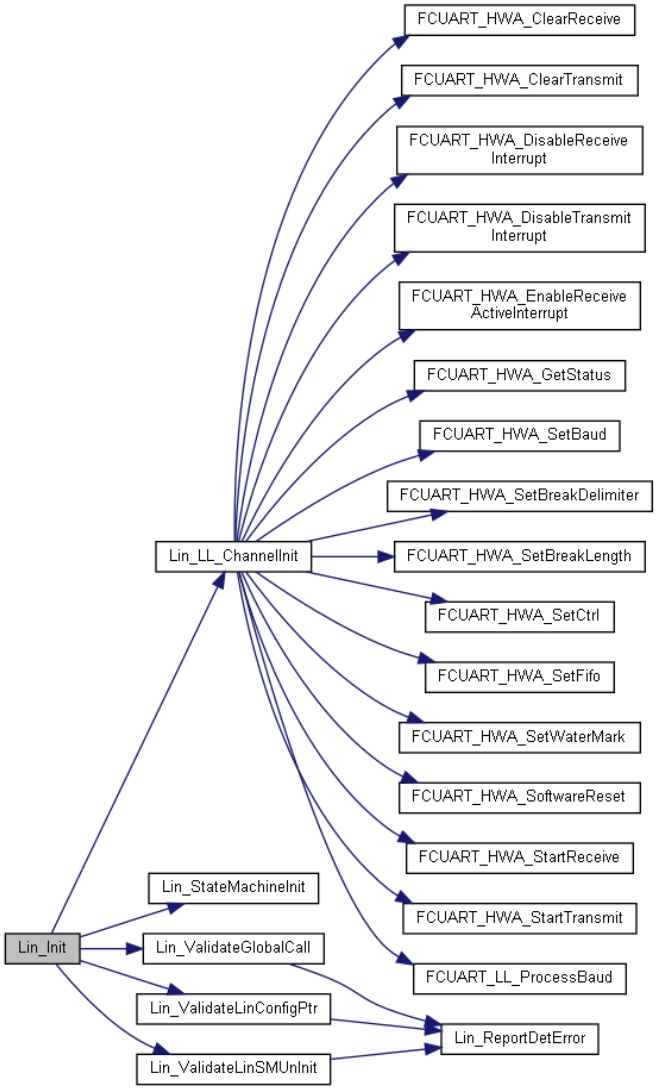
2.6.4 Lin_PduType

Structure	Lin_PduType
Description	The LIN identifier (0..0x3F) with its parity bits(bit6,bit7).
Diagram	N/A
Data Fields	• Lin_FramePidType Pid • LIN frame identifier. • Lin_FrameCsModelType Cs • Checksum model type. • Lin_FrameResponseType Drc • Response type. • Lin_FrameDlType Dl • Data length. • uint8* SduPtr • Pointer to Sdu.

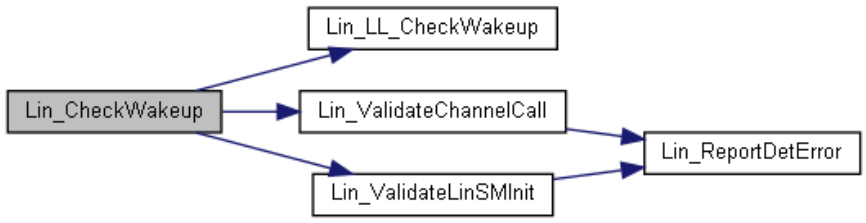
2.7 API Functions

2.7.1 Functions in Lin.h

2.7.1.1 void Lin_Init(const Lin_ConfigType* Config)

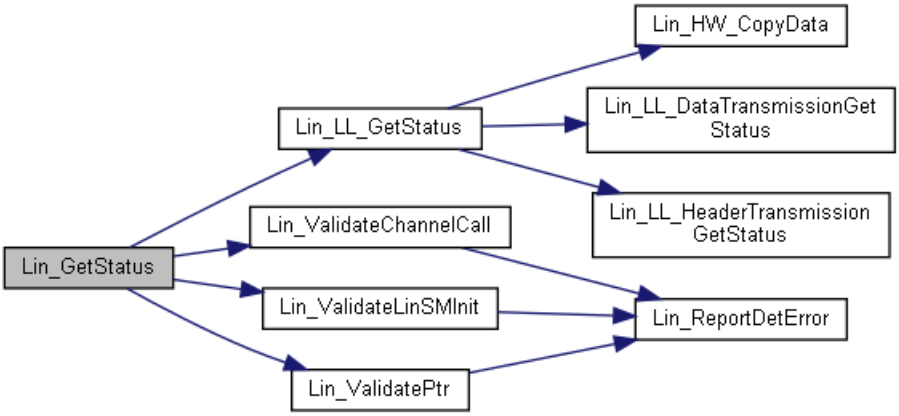
Function	void Lin_Init(const Lin_ConfigType* Config)	
Description	Initializes the Lin module.	
Diagram		
Parameters	**Parameter**	**Description**
	Config	Init configuration.
Returns	N/A	

2.7.1.2 Std_ReturnType Lin_CheckWakeup(uint8 Channel)

Function	Std_ReturnType Lin_CheckWakeup(uint8 Channel)	
Description	Check the wake up of Lin channel. This function identifies if the Lin channel has been woken up.	
Diagram		
Parameters	**Parameter**	**Description**

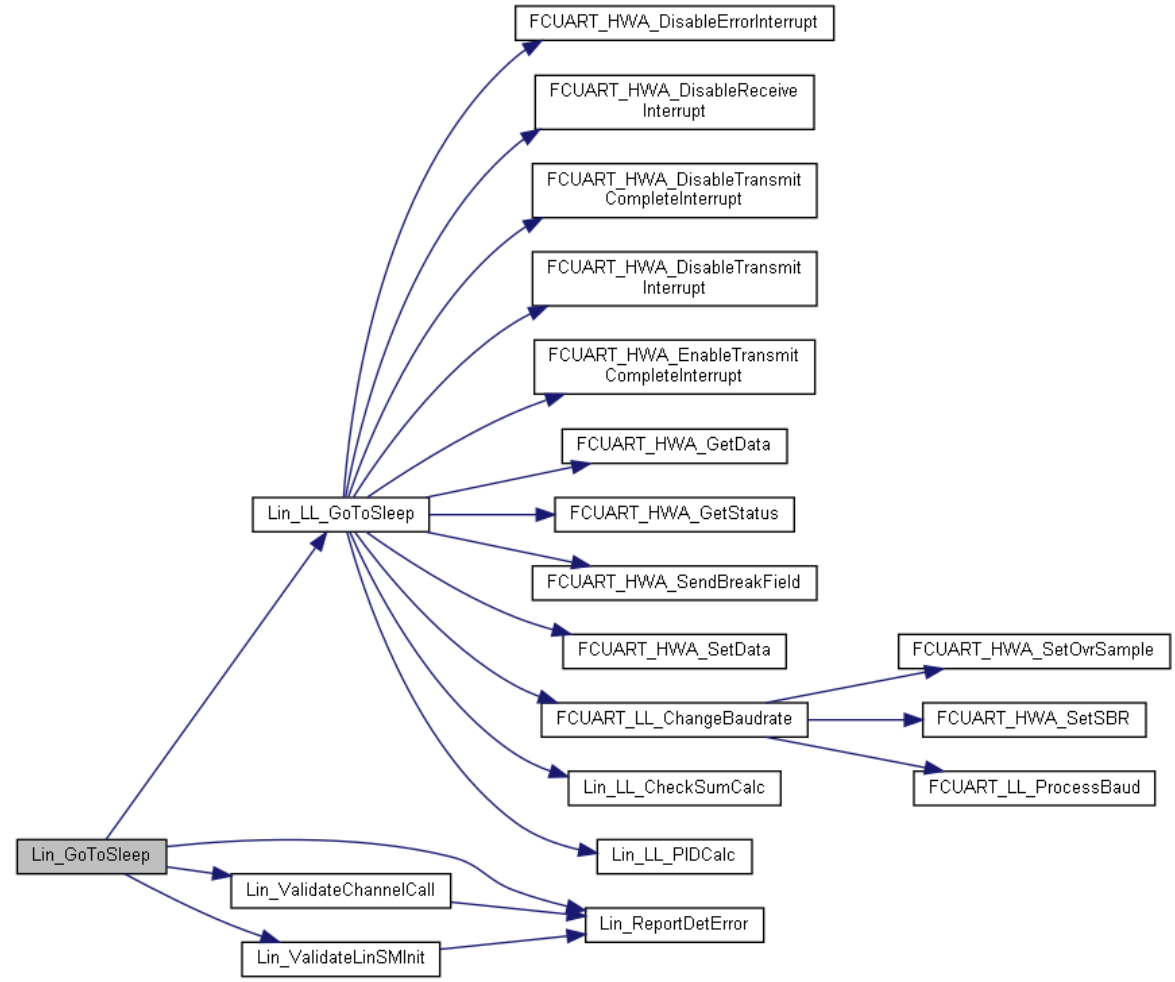
	Channel	LIN channel to be addressed.
Returns	Std_ReturnType	

2.7.1.3 Lin_StatusType Lin_GetStatus(uint8 Channel, const uint8 **Lin_SduPtr);

Function	Lin_StatusType Lin_GetStatus(uint8 Channel, const uint8 **Lin_SduPtr);	
Description	Gets the status of the LIN driver. This function returns the state of the current transmission, reception or operation status. If the reception of a Slave response was successful then this service provides a pointer to the buffer where the data is stored.	
Diagram		
Parameters	**Parameter**	**Description**
	Channel	LIN channel to be addressed.
	Lin_SduPtr	Status of LIN channel.
Returns	Lin_StatusType	

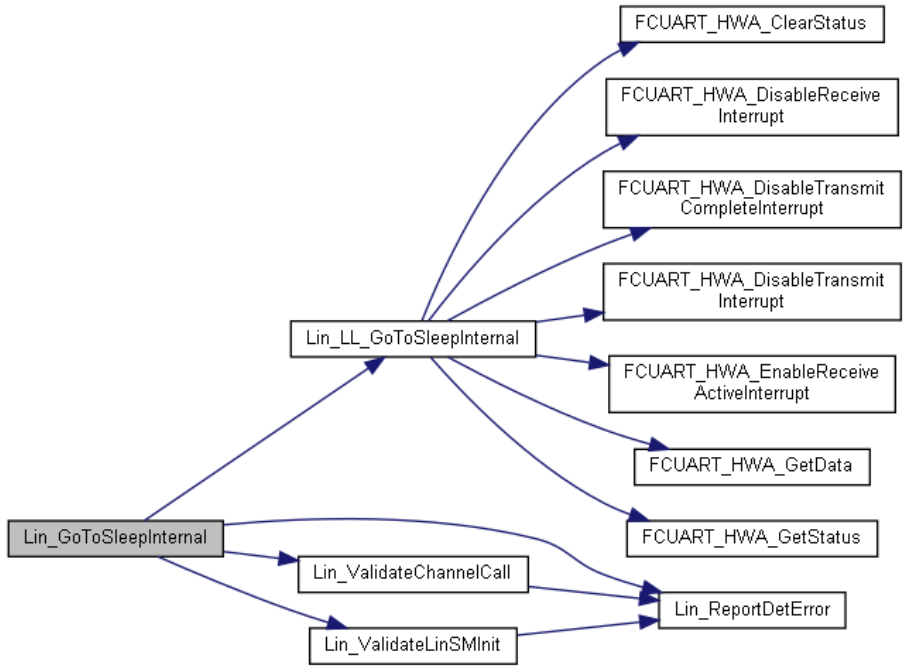
2.7.1.4 Std_ReturnType Lin_GoToSleep(uint8 Channel)

Function	Std_ReturnType Lin_GoToSleep(uint8 Channel)
Description	The service instructs the driver to transmit a go-to-sleep-command on the addressed Lin channel. This function stops any ongoing transmission and initiates the transmission of the sleep command (master command frame with ID = 0x3C and data = (0x00, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF). State transition in LIN_CH_SLEEP_STATE shall be done after the completion of the sleep command transmission regardless of the success (therefore the ISR is responsible to put the channel in LIN_CH_SLEEP_STATE).

Diagram		
Parameters	**Parameter**	**Description**
	Channel	LIN channel to be addressed.
Returns	Std_ReturnType	

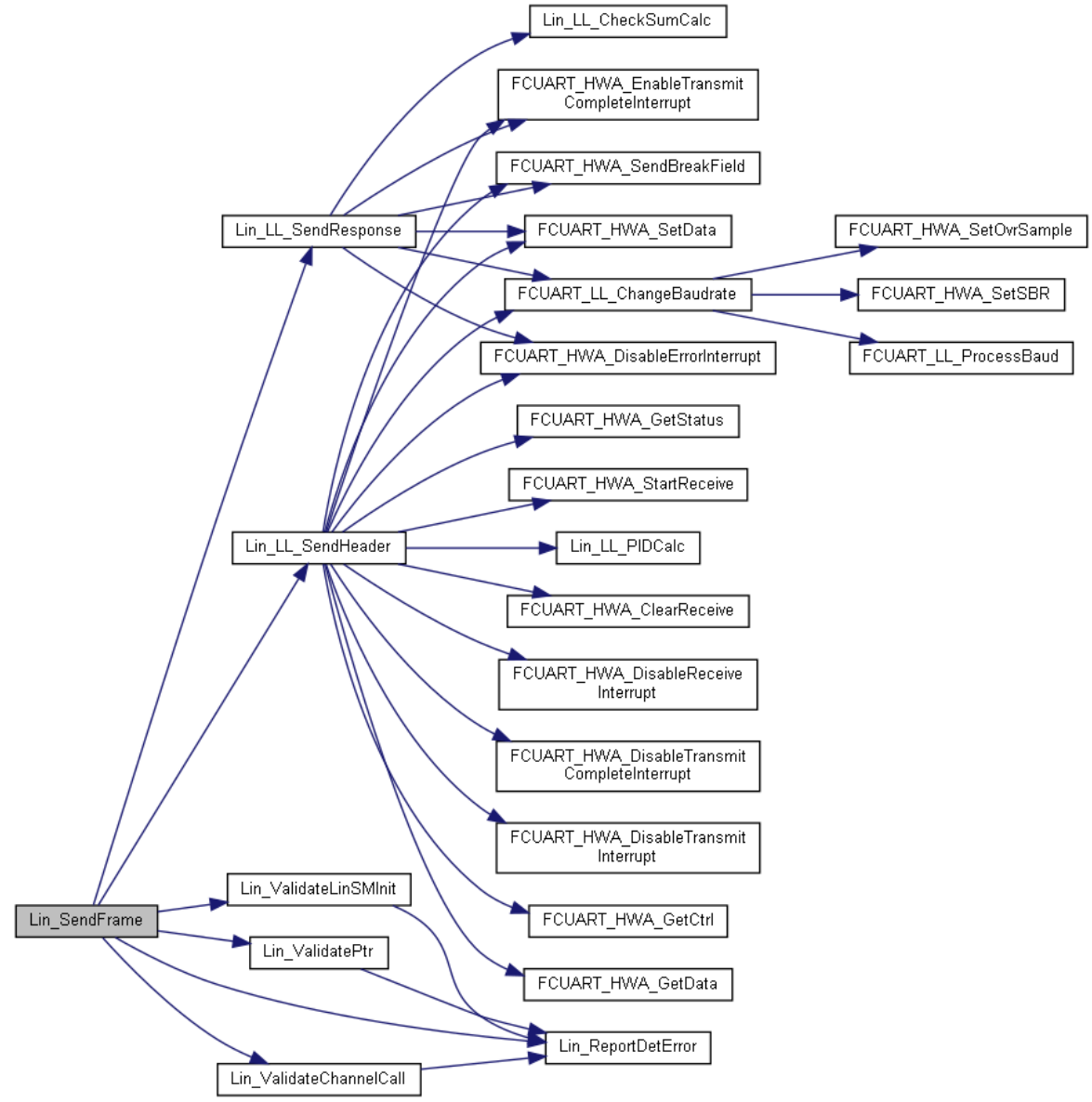
2.7.1.5 Std_ReturnType Lin_GoToSleepInternal(uint8 Channel)

Function	Std_ReturnType Lin_GoToSleepInternal(uint8 Channel)
Description	Put a Lin channel in the internal sleep state. Stops any ongoing transmission, sets the channel state to LIN_CH_SLEEP and put the LIN hardware unit to a reduced power operation mode.

Diagram		
Parameters	**Parameter**	**Description**
	Channel	LIN channel to be addressed.
Returns	Std_ReturnType	

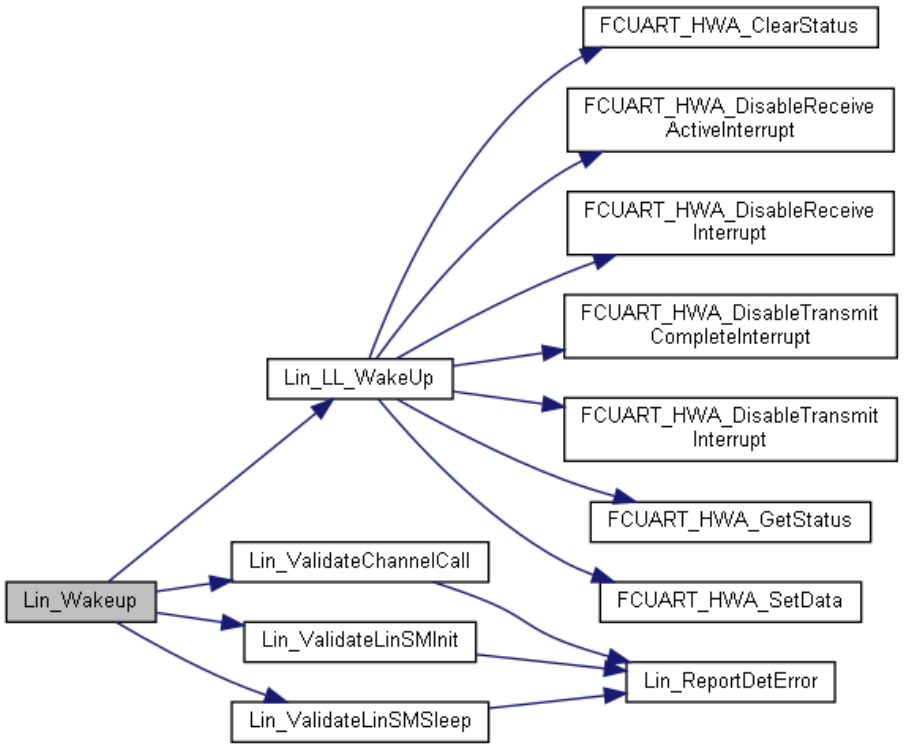
2.7.1.6 Std_ReturnType Lin_SendFrame(uint8 Channel, const Lin_PduType *PduInfoPtr)

Function	Std_ReturnType Lin_SendFrame(uint8 Channel, const Lin_PduType *PduInfoPtr)
Description	Sends a Lin frame. Sends a Lin header and a Lin response, if necessary. The direction of the frame response (master response, slave response, slave-to-slave communication) is provided by the PduInfoPtr.

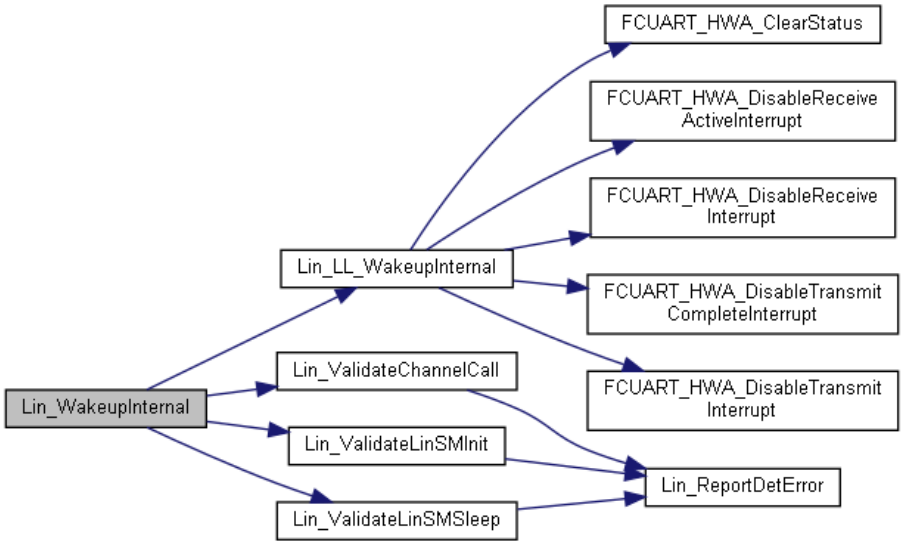
Diagram		
Parameters	**Parameter**	**Description**
	Channel	LIN channel to be addressed.
	PduInfoPtr	Pointer of PDU information.
Returns	Std_ReturnType	

2.7.1.7 Std_ReturnType Lin_Wakeup(uint8 Channel)

Function	Std_ReturnType Lin_Wakeup(uint8 Channel)
Description	Generates a wake up pulse. This function shall send a wake up signal to the LIN bus and put the Lin channel in LIN_CH_OPERATIONAL state.

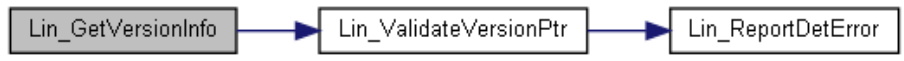
Diagram		
Parameters	**Parameter**	**Description**
	Channel	LIN channel to be addressed.
Returns	Std_ReturnType	

2.7.1.8 Std_ReturnType Lin_WakeupInternal(uint8 Channel)

Function	Std_ReturnType Lin_WakeupInternal(uint8 Channel)	
Description	Wake up the Lin channel. This function shall put the LIN channel in LIN_CH_OPERATIONAL state without sending a wake up signal to the LIN bus.	
Diagram		
Parameters	**Parameter**	**Description**
	Channel	LIN channel to be addressed.
Returns	Std_ReturnType	

2.7.1.9 void Lin_GetVersionInfo (Std_VersionInfoType* versioninfo)

Function	void Lin_GetVersionInfo (Std_VersionInfoType* versioninfo)

Description	Returns the version information of this module.	
Diagram		
Parameters	**Parameter**	**Description**
	versioninfo	Pointer to where to store the version information of this module.
Returns	N/A	

2.8 Hardware Functions

2.8.1 Functions in Lin_FCUart.h

2.8.1.1 void Lin_LL_ChannelInit(uint8 u8Channel, const Lin_CoreConfigType *pConfig)

Function	void Lin_LL_ChannelInit(uint8 u8Channel, const Lin_CoreConfigType *pConfig)	
Description	This function initializes hardware channel through HardWareAbsract(HWA).	
Parameters	**Parameter**	**Description**
	u8Channel	LIN channel to be addressed.
	pConfig	Lin configuration.
Returns	N/A	
Referenced By	Lin_Init()	

2.8.1.2 Std_ReturnType Lin_LL_CheckWakeup(uint8 Channel)

Function	Std_ReturnType Lin_LL_CheckWakeup(uint8 Channel)	
Description	Check if a LIN channel has been waked-up. This function identifies if the addressed LIN channel has been woken up by the LIN bus transceiver. It checks the wake up flag from the addressed LIN channel which must be in sleep mode and have the wake up signal.	
Parameters	**Parameter**	**Description**
	Channel	LIN channel to be addressed.
Returns	Std_ReturnType	
Referenced By	Lin_ CheckWakeup()	

2.8.1.3 Lin_StatusType Lin_LL_GetStatus(const uint8 u8Channel, uint8 *pu8LinSdu)

Function	Lin_StatusType Lin_LL_GetStatus(const uint8 u8Channel, uint8 *pu8LinSdu)	
Description	Gets the status of the LIN driver when Channel is operating. . This function returns the state of the current transmission, reception or operation status. If the reception of a Slave response was successful then this service provides a pointer to the buffer where the data is stored.	
Parameters	**Parameter**	**Description**
	u8Channel	LIN channel to be addressed.
	pu8LinSdu	Pointer to pointer to a shadow buffer or memory mapped LIN Hardware receive buffer where the current SDU is stored.
Returns	Lin_StatusType.	
Referenced By	Lin_GetStatus()	

2.8.1.4 Std_ReturnType Lin_LL_GoToSleep(uint8 u8Channel, uint8 u8Module)

Function	Std_ReturnType Lin_LL_GoToSleep(uint8 u8Channel, uint8 u8Module)	
Description	This function stops any ongoing transmission and initiates the transmission of the sleep command (master command frame with ID = 0x3C and data = (0x00, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF).	
Parameters	**Parameter**	**Description**
	u8Channel	LIN channel to be addressed.
	pu8LinSdu	Pointer to pointer to a shadow buffer or memory mapped LIN Hardware receive buffer where the current SDU is stored.
Returns	Lin_StatusType.	
Referenced By	Lin_GetStatus()	

2.8.1.5 Std_ReturnType Lin_LL_GoToSleepInternal(uint8 u8Channel, uint8 u8Module)

Function	Std_ReturnType Lin_LL_GoToSleepInternal(uint8 u8Channel, uint8 u8Module)	
Description	This function stops any ongoing transmission and put the Channel in sleep mode (then Lin hardware enters a reduced power operation mode).	
Parameters	**Parameter**	**Description**
	u8Channel	LIN channel to be addressed.
Returns	Std_ReturnType.	
Referenced By	Lin_GotoSleepInternal ()	

2.8.1.6 Std_ReturnType Lin_LL_SendHeader(uint8 u8Channel,uint8 u8Module, const Lin_PduType *pPduInfoPtr)

Function	Std_ReturnType Lin_LL_SendHeader(uint8 u8Channel,uint8 u8Module, const Lin_PduType *pPduInfoPtr)	
Description	Sends the header part of the Lin frame.Initiates the transmission of the header part of the Lin frame on Channel using information stored on PduInfoPtr pointer. If response type is MASTER_RESPONSE then nothing is sent over the bus the entire frame (including header) is sent with the Lin_LL_SendResponse	
Parameters	**Parameter**	**Description**
	u8Channel	LIN channel to be addressed.
	u8Module	Hardware ID.
	pPduInfoPtr	Pointer to PDU containing the PID, Checksum model, Response type, Dl and SDU data pointer.
Returns	Std_ReturnType.	
Referenced By	Lin_SendFrame()	

2.8.1.7 void Lin_LL_SendResponse(uint8 u8Channel, uint8 u8Module, const Lin_PduType *pPduInfoPtr)

Function	void Lin_LL_SendResponse(uint8 u8Channel, uint8 u8Module, const Lin_PduType *pPduInfoPtr)	
Description	Initiates the transmission of the data part of the Lin frame on Channel using information stored on PduInfoPtr pointer.	
Parameters	**Parameter**	**Description**
	u8Channel	LIN channel to be addressed.
	u8Module	Hardware ID.

	pPduInfoPtr	Pointer to PDU containing the PID, Checksum model, Response type, Dl and SDU data pointer.
Returns	Std_ReturnType.	
Referenced By	Lin_SendFrame()	

2.8.1.8 void Lin_LL_WakeUp(uint8 u8Channel, uint8 u8Module)

Function	void Lin_LL_WakeUp(uint8 u8Channel, uint8 u8Module)	
Description	This function shall send a wake up signal to the LIN bus and put the Lin channel in LIN_CH_OPERATIONAL state.	
Parameters	**Parameter**	**Description**
	u8Channel	LIN channel to be addressed.
Returns	N/A	
Referenced By	Lin_WakeUp ()	

2.8.1.9 void Lin_LL_WakeupInternal(uint8 u8Channel, uint8 u8Module)

Function	void Lin_LL_WakeupInternal(uint8 u8Channel, uint8 u8Module)	
Description	This function shall put the Lin channel in LIN_CH_OPERATIONAL state without sending a wake up signal to the Lin bus.	
Parameters	**Parameter**	**Description**
	u8Channel	LIN channel to be addressed.
Returns	N/A	
Referenced By	Lin_WakeupInternal()	

2.8.1.10 void Lin_LL_TxRxInterruptHandler(const uint8 u8Module);

Function	void Lin_LL_TxRxInterruptHandler(const uint8 u8Module);	
Description	This function shall manage all the RX and TX ISRs on the addressed channel.	
Parameters	**Parameter**	**Description**
	u8Module	Uart hardware ID.
Returns	N/A	
Referenced By	N/A	

2.8.1.11 void Lin_LL_RxDataReadyHandler(uint8 u8Module, uint8 u8Channel)

Function	void Lin_LL_RxDataReadyHandler(uint8 u8Module, uint8 u8Channel)	
Description	This function shall manage the RX ISRs on the addressed channel.	
Parameters	**Parameter**	**Description**
	u8Module	Uart hardware ID.
Returns	N/A	
Referenced By	N/A	

2.8.1.12 void Lin_LL_InterruptSourceSlave(uint8 u8Module, uint8 u8Channel, uint32 u32Data)

Function	void Lin_LL_InterruptSourceSlave(uint8 u8Module, uint8 u8Channel, uint32 u32Data)

Description	This function shall manage the RX ISRs on the addressed channel when the frame command is a slave response command.	
Parameters	**Parameter**	**Description**
	u8Module	Uart hardware ID.
Returns	N/A	
Referenced By	N/A	

2.8.1.13 void Lin_LL_IdleInterruptHandler(uint8 u8Module, uint8 u8Channel)

Function	void Lin_LL_IdleInterruptHandler(uint8 u8Module, uint8 u8Channel)	
Description	This function shall manage the RX ISRs on the addressed channel.	
Parameters	**Parameter**	**Description**
	u8Module	Uart hardware ID.
Returns	N/A	
Referenced By	N/A	

2.8.1.14 void Lin_LL_ErrorInterruptHandler(const uint8 u8Module)

Function	void Lin_LL_ErrorInterruptHandler(const uint8 u8Module)	
Description	This function shall manage all the Error ISRs on the addressed channel.	
Parameters	**Parameter**	**Description**
	u8Module	Uart hardware ID.
Returns	N/A	
Referenced By	N/A	

2.9 Peripheral Functions

2.9.1 Functions in Lin_FCUart_RegOps.h

2.9.1.1 LOCAL_INLINE uint32 FCUART_HWA_GetStatus(FCUART_Type *pUart, FCUART_StatType eStatusType)

Function	LOCAL_INLINE uint32 FCUART_HWA_GetStatus(FCUART_Type *pUart, FCUART_StatType eStatusType)	
Description	Get Stat Flag.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	eStatusType	Status type.
Returns	FCUART STAT status flag	
Referenced By	N/A	

2.9.1.2 LOCAL_INLINE uint32 FCUART_HWA_GetInterruptStatus(FCUART_Type *pUart, FCUART_Ctrl_IEStaType eStatusType)

Function	LOCAL_INLINE uint32 FCUART_HWA_GetInterruptStatus(FCUART_Type *pUart, FCUART_Ctrl_IEStaType eStatusType)	
Description	Get Ctrl Interrupt Status.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.

	eStatusType	Interrupt stat type.
Returns	FCUART CTRL interrupt status flag	

2.9.1.3 LOCAL_INLINE void FCUART_HWA_ClearStatus(FCUART_Type *pUart, FCUART_StatType eStatusType)

Function	LOCAL_INLINE void FCUART_HWA_ClearStatus(FCUART_Type *pUart, FCUART_StatType eStatusType)	
Description	Clear Stat Flag.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	eStatusType	Status type.
Returns	N/A	
Referenced By	N/A	

2.9.1.4 LOCAL_INLINE void FCUART_HWA_StartTransmit(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_StartTransmit(FCUART_Type *pUart)	
Description	Start transmit.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.5 LOCAL_INLINE void FCUART_HWA_ClearTransmit(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_ClearTransmit(FCUART_Type *pUart)	
Description	Clear transmit.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value
Returns	N/A	
Referenced By	N/A	

2.9.1.6 LOCAL_INLINE void FCUART_HWA_StartReceive(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_StartReceive(FCUART_Type *pUart)	
Description	Start receive.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.7 LOCAL_INLINE void FCUART_HWA_ClearReceive(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_ClearReceive(FCUART_Type *pUart)	
Description	clear receive.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	

Referenced By	N/A

2.9.1.8 LOCAL_INLINE void FCUART_HWA_EnableReceiveInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_EnableReceiveInterrupt(FCUART_Type *pUart)	
Description	Enable Receive Interrupt.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.9 LOCAL_INLINE void FCUART_HWA_DisableReceiveInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_DisableReceiveInterrupt(FCUART_Type *pUart)	
Description	Disable Receive Interrupt.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.10 LOCAL_INLINE void FCUART_HWA_EnableTransmitInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_EnableTransmitInterrupt(FCUART_Type *pUart)	
Description	Enable Transmit Interrupt.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.11 LOCAL_INLINE void FCUART_HWA_DisableTransmitInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_DisableTransmitInterrupt(FCUART_Type *pUart)	
Description	Disable Transmit Interrupt.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.12 LOCAL_INLINE void FCUART_HWA_EnableTransmitCompleteInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_EnableTransmitCompleteInterrupt(FCUART_Type *pUart)	
Description	Enable Transmit Complete Interrupt.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.13 LOCAL_INLINE void FCUART_HWA_DisableTransmitCompleteInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_DisableTransmitCompleteInterrupt(FCUART_Type *pUart)	
Description	Disable Transmit Complete Interrupt.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.14 LOCAL_INLINE void FCUART_HWA_EnableErrorInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_EnableErrorInterrupt(FCUART_Type *pUart)	
Description	Enable Error Interrupt(Parity Error Exclude).	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.15 LOCAL_INLINE void FCUART_HWA_DisableErrorInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_DisableErrorInterrupt(FCUART_Type *pUart)	
Description	Disable Error Interrupt(Parity Error Exclude).	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.16 LOCAL_INLINE void FCUART_HWA_SetCtrl(FCUART_Type *pUart, uint32 u32Value)

Function	LOCAL_INLINE void FCUART_HWA_SetCtrl(FCUART_Type *pUart, uint32 u32Value)	
Description	Set FCUART Ctrl register.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	u32Value	Written value.
Returns	N/A	
Referenced By	N/A	

2.9.1.17 LOCAL_INLINE void FCUART_HWA_AttachCtrl(FCUART_Type *pUart, uint32 u32Value)

Function	LOCAL_INLINE void FCUART_HWA_AttachCtrl(FCUART_Type *pUart, uint32 u32Value)	
Description	Attach FCUART Ctrl register.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	u32Value	Written value.
Returns	N/A	
Referenced By	N/A	

2.9.1.18 LOCAL_INLINE void FCUART_HWA_SetBreakDelimiter (FCUART_Type *pUart, Lin_BreakDelimiterType eBreakLen)

Function	LOCAL_INLINE void FCUART_HWA_SetBreakDelimiter (FCUART_Type *pUart, Lin_BreakDelimiterType eBreakLen)	
Description	Set FCUART Break Delimiter length.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	eBreakLen	Break delimiter length.
Returns	N/A	
Referenced By	N/A	

2.9.1.19 LOCAL_INLINE void FCUART_HWA_SetBaud(FCUART_Type *pUart, uint32 u32Value)

Function	LOCAL_INLINE void FCUART_HWA_SetBaud(FCUART_Type *pUart, uint32 u32Value)	
Description	Set FCUART Baud register.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	u32Value	Written value.
Returns	N/A	
Referenced By	N/A	

2.9.1.20 LOCAL_INLINE void FCUART_HWA_AttachBaud(FCUART_Type *pUart, uint32 u32Value)

Function	LOCAL_INLINE void FCUART_HWA_AttachBaud(FCUART_Type *pUart, uint32 u32Value)	
Description	Attach FCUART Baud register.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	u32Value	Written value.
Returns	N/A	
Referenced By	N/A	

2.9.1.21 LOCAL_INLINE void FCUART_HWA_SetFifo(FCUART_Type *pUart, uint32 u32Value)

Function	LOCAL_INLINE void FCUART_HWA_SetFifo(FCUART_Type *pUart, uint32 u32Value)	
Description	Set FCUART Fifo register.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	u32Value	Written value.
Returns	N/A	
Referenced By	N/A	

2.9.1.22 LOCAL_INLINE void FCUART_HWA_AttachFifo(FCUART_Type *pUart, uint32 u32Value)

Function	LOCAL_INLINE void FCUART_HWA_AttachFifo(FCUART_Type *pUart, uint32 u32Value)	
Description	Attach FCUART Fifo register.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.

	u32Value	Written value.
Returns	N/A	
Referenced By	N/A	

2.9.1.23 LOCAL_INLINE void FCUART_HWA_SetWaterMark(FCUART_Type *pUart, uint32 u32Value)

Function	LOCAL_INLINE void FCUART_HWA_SetWaterMark(FCUART_Type *pUart, uint32 u32Value)	
Description	Set FCUART WaterMark register.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	u32Value	Written value.
Returns	N/A	
Referenced By	N/A	

2.9.1.24 LOCAL_INLINE void FCUART_HWA_AttachWaterMark(FCUART_Type *pUart, uint32 u32Value)

Function	LOCAL_INLINE void FCUART_HWA_AttachWaterMark(FCUART_Type *pUart, uint32 u32Value)	
Description	Attach FCUART WaterMark register.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	u32Value	Written value.
Returns	N/A	
Referenced By	N/A	

2.9.1.25 LOCAL_INLINE void FCUART_HWA_SetMatch(FCUART_Type *pUart, uint32 u32Value)

Function	LOCAL_INLINE void FCUART_HWA_SetMatch(FCUART_Type *pUart, uint32 u32Value)	
Description	Set FCUART Match register.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	u32Value	Written value.
Returns	N/A	
Referenced By	N/A	

2.9.1.26 LOCAL_INLINE void FCUART_HWA_AttachMatch(FCUART_Type *pUart, uint32 u32Value)

Function	LOCAL_INLINE void FCUART_HWA_AttachMatch(FCUART_Type *pUart, uint32 u32Value)	
Description	Attach FCUART Match register.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	u32Value	Written value.
Returns	N/A	
Referenced By	N/A	

2.9.1.27 LOCAL_INLINE uint32 FCUART_HWA_ReadSTAT(FCUART_Type *pUart)

Function	LOCAL_INLINE uint32 FCUART_HWA_ReadSTAT(FCUART_Type *pUart)

Description	Read FCUART STAT register.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	STAT read value	
Referenced By	N/A	

2.9.1.28 LOCAL_INLINE void FCUART_HWA_WriteClesarSTAT(FCUART_Type *pUart, uint32 u32Value)

Function	LOCAL_INLINE void FCUART_HWA_WriteClesarSTAT(FCUART_Type *pUart, uint32 u32Value)	
Description	Write 1 Clear FCUART STAT register.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	u32Value	Written value.
Returns	N/A	
Referenced By	N/A	

2.9.1.29 LOCAL_INLINE void FCUART_HWA_SetBitModeAndParity(FCUART_Type *pUart, FCUART_BitModeType eBitMode, uint8 bParityEnable, FCUART_ParityType eParityType,

Function	LOCAL_INLINE void FCUART_HWA_SetBitModeAndParity(FCUART_Type *pUart, FCUART_BitModeType eBitMode, uint8 bParityEnable, FCUART_ParityType eParityType,	
Description	Set Bit Mode and Parity.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	eBitMode	Bit mode, 8 or 9 bits
	bParityEnable	If enable Parity, set 1U, or set 0U.
	eParityType	Parity type, odd or even.
	eStopBit	Stop bits number, 1 or 2 bits.
Returns	N/A	
Referenced By	N/A	

2.9.1.30 LOCAL_INLINE void FCUART_HWA_EnableReceiveDMA(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_EnableReceiveDMA(FCUART_Type *pUart)	
Description	Enable Receive DMA.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.31 LOCAL_INLINE void FCUART_HWA_DisableReceiveDMA(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_DisableReceiveDMA(FCUART_Type *pUart)	
Description	Disable Receive DMA.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.

Returns	N/A
Referenced By	N/A

2.9.1.32 LOCAL_INLINE void FCUART_HWA_EnableReceiveFIFO(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_EnableReceiveFIFO(FCUART_Type *pUart)	
Description	Enable Receive FIFO.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.33 LOCAL_INLINE void FCUART_HWA_DisableReceiveFIFO(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_DisableReceiveFIFO(FCUART_Type *pUart)	
Description	Disable Receive FIFO.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.34 LOCAL_INLINE void FCUART_HWA_SetData(FCUART_Type *pUart, uint32 u32Data)

Function	LOCAL_INLINE void FCUART_HWA_SetData(FCUART_Type *pUart, uint32 u32Data)	
Description	Set the data value.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	u32Data	Set data.
Returns	N/A	
Referenced By	N/A	

2.9.1.35 LOCAL_INLINE uint8 FCUART_HWA_GetData(FCUART_Type *pUart)

Function	LOCAL_INLINE uint8 FCUART_HWA_GetData(FCUART_Type *pUart)	
Description	Get the data value.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	The data value.	
Referenced By	N/A	

2.9.1.36 LOCAL_INLINE uint32 FCUART_HWA_GetDataRegStatus(FCUART_Type *pUart)

Function	LOCAL_INLINE uint32 FCUART_HWA_GetDataRegStatus(FCUART_Type *pUart)	
Description	Get the data register value.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	The data register value.	

Referenced By	N/A

2.9.1.37 LOCAL_INLINE void FCUART_HWA_SoftwareReset(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_SoftwareReset(FCUART_Type *pUart)	
Description	Reset the instance by software.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.38 LOCAL_INLINE void FCUART_HWA_DisableBreakDetectInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_DisableBreakDetectInterrupt(FCUART_Type *pUart)	
Description	Disable LIN break detect interrupt.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.39 LOCAL_INLINE void FCUART_HWA_EnableBreakDetectInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_EnableBreakDetectInterrupt(FCUART_Type *pUart)	
Description	Enable LIN break detect interrupt.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.40 LOCAL_INLINE void FCUART_HWA_SetBreakLength(FCUART_Type *pUart, Lin_BreakLengthType eBreakLen)

Function	LOCAL_INLINE void FCUART_HWA_SetBreakLength(FCUART_Type *pUart, Lin_BreakLengthType eBreakLen)	
Description	Disable LIN break Length.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	Lin_BreakLengthType	eBreakLen.
Returns	N/A	
Referenced By	N/A	

2.9.1.41 LOCAL_INLINE void FCUART_HWA_SendBreakField(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_SendBreakField(FCUART_Type *pUart)	
Description	Send a LIN break field.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	

Referenced By	N/A

2.9.1.42 LOCAL_INLINE void FCUART_HWA_EnableReceiveActiveInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_EnableReceiveActiveInterrupt(FCUART_Type *pUart)	
Description	Enable UART receive active interrupt.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.43 LOCAL_INLINE void FCUART_HWA_DisableReceiveActiveInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_DisableReceiveActiveInterrupt(FCUART_Type *pUart)	
Description	Disable UART receive active interrupt.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.9.1.44 LOCAL_INLINE uint32 FCUART_HWA_GetReceiveActiveInterruptStatus(FCUART_Type *pUart)

Function	LOCAL_INLINE uint32 FCUART_HWA_GetReceiveActiveInterruptStatus(FCUART_Type *pUart)	
Description	Get UART receive active interrupt status.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	return	FALSE for disable, TRUE for enable.
Returns	N/A	
Referenced By	N/A	

2.9.1.45 LOCAL_INLINE void FCUART_HWA_SetIdleConfig(FCUART_Type *pUart, Lin_IdleConfigType eType)

Function	LOCAL_INLINE void FCUART_HWA_SetIdleConfig(FCUART_Type *pUart, Lin_IdleConfigType eType)	
Description	Set UART receive active interrupt status.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
	uint8	u8Data to be configured.
Returns	N/A	
Referenced By	N/A	

2.9.1.46 LOCAL_INLINE void FCUART_HWA_EnableIdleInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_EnableIdleInterrupt(FCUART_Type *pUart)	
Description	Enable UART idle line interrupt.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	

Referenced By	N/A

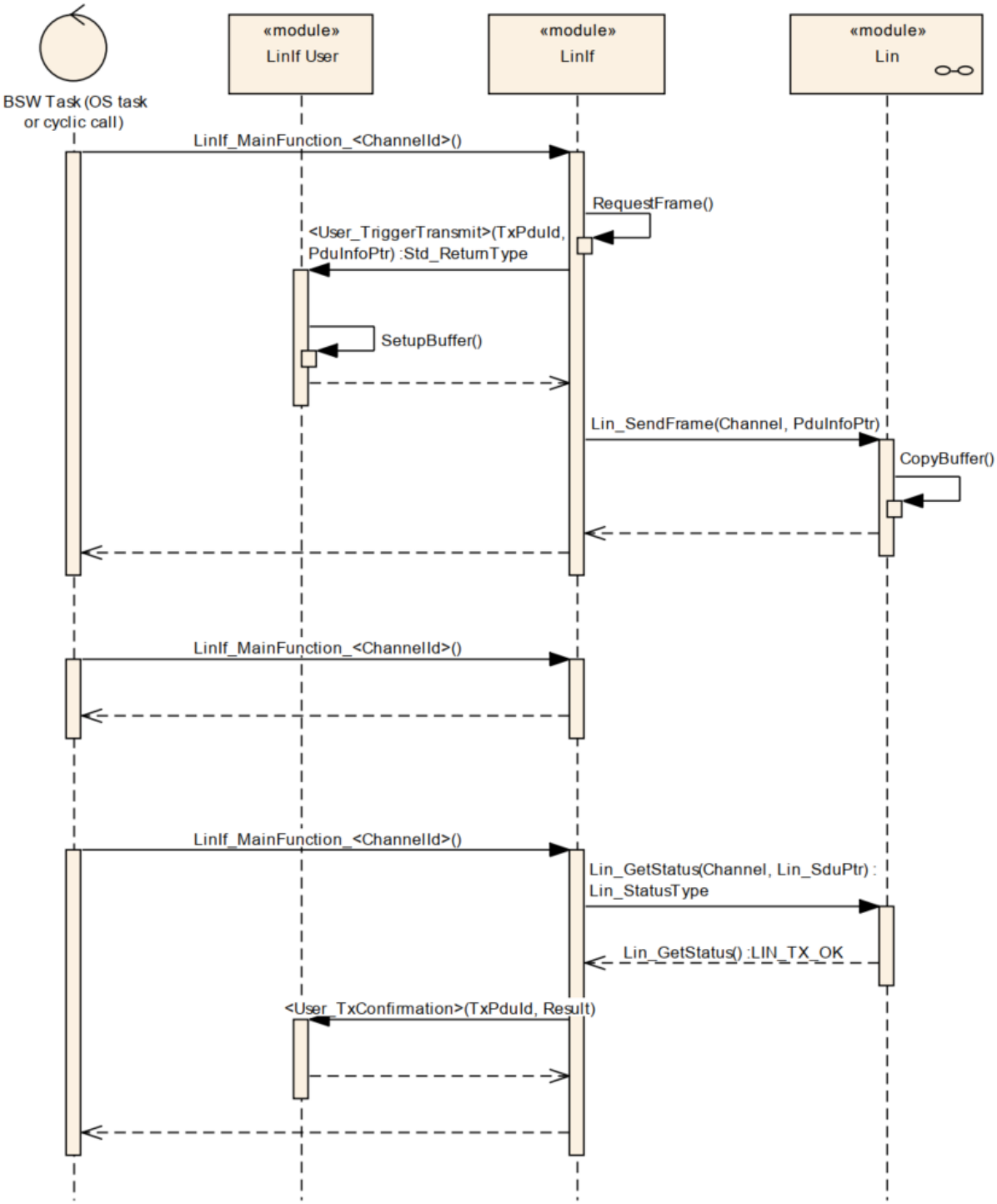
2.9.1.47 LOCAL_INLINE void FCUART_HWA_DisableIdleInterrupt(FCUART_Type *pUart)

Function	LOCAL_INLINE void FCUART_HWA_DisableIdleInterrupt(FCUART_Type *pUart)	
Description	Disable UART idle line interrupt.	
Parameters	**Parameter**	**Description**
	pUart	UART instance value.
Returns	N/A	
Referenced By	N/A	

2.10 API Sequence Diagram

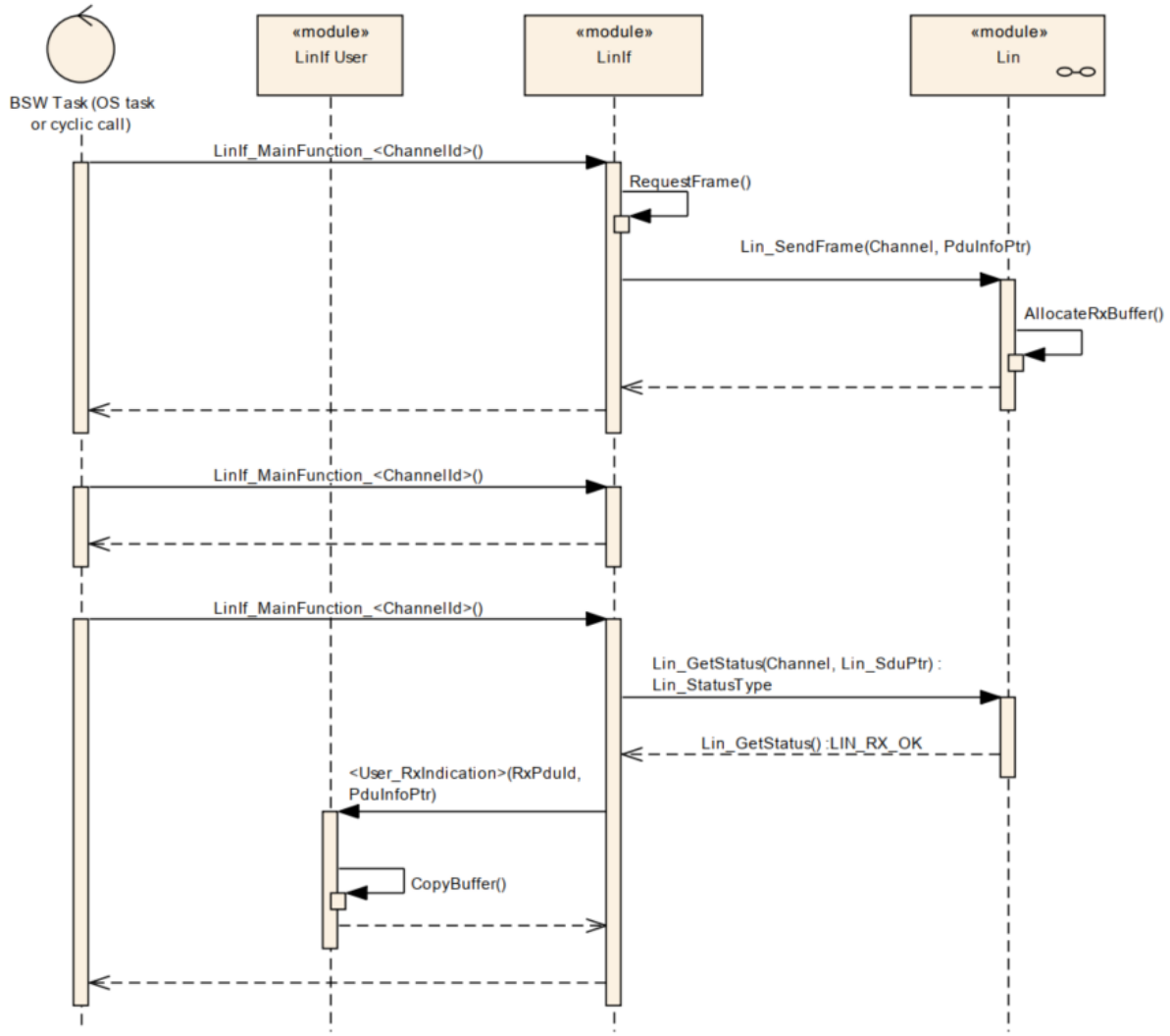
2.10.1 Frame Transmission

The following use case shows the transmission of a LIN frame. The first call of the LinIf_MainFunction_<ChannelId> requests transmission of the header and the response. During the second call, the frame is under transmission. In the third call of the LinIf_MainFunction_<ChannelId>, the frame is finished. The RequestFrame call in the diagram is the interface call to the Schedule Table Manager. The LinIf_MainFunction_<ChannelId> gets the frame to send and the delay to the next frame. The CopyBuffer call is to show that the copying of the SDU is made in the LIN Driver and not in the LIN Interface.



2.10.2 Frame Reception

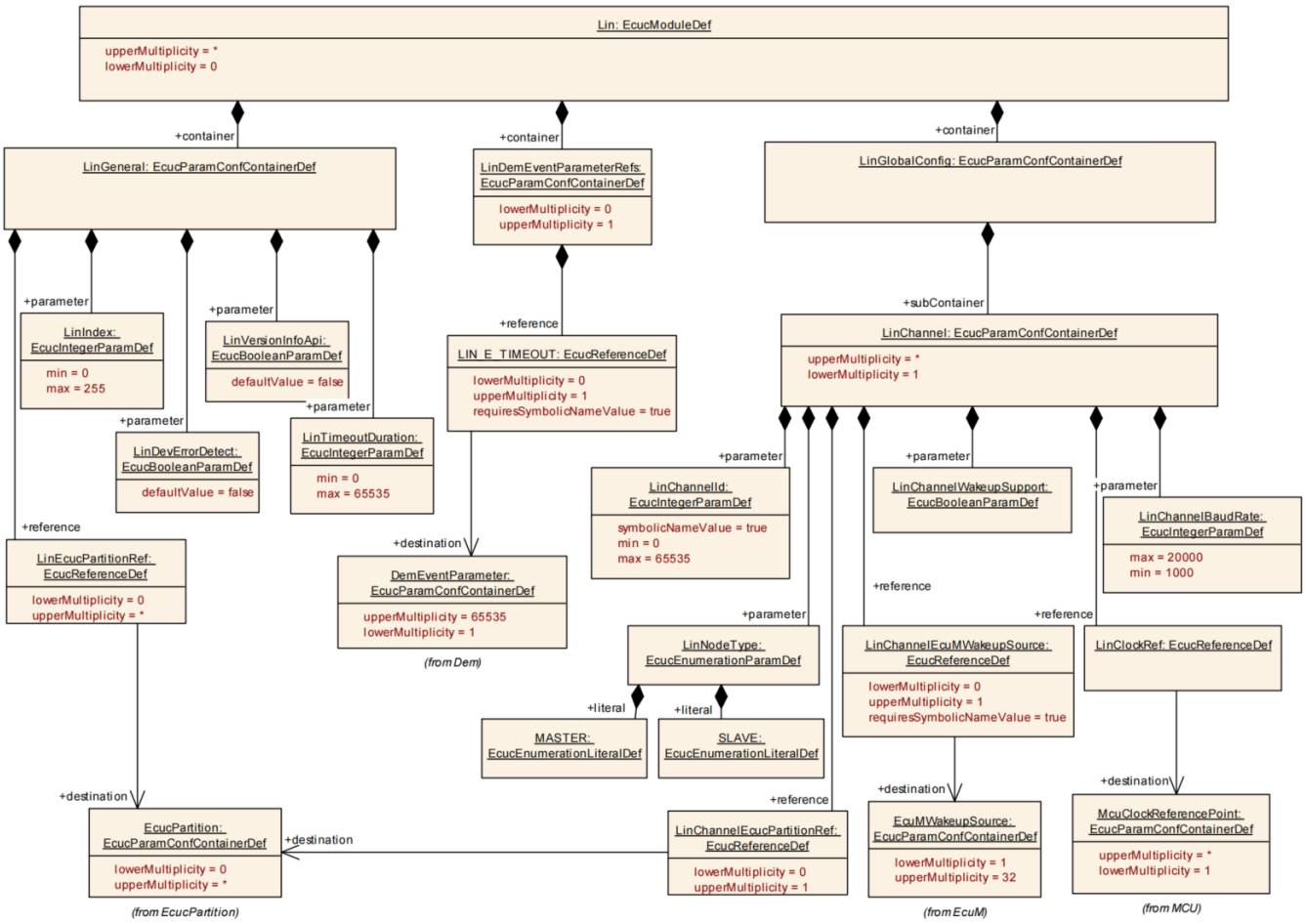
The following use case shows the reception of a LIN frame. The first call of the LinIf_MainFunction_<ChannelId> requests transmission of the header. During the second call, the frame is under transmission. In the third call, the frame is finished (this call is called after the maximum frame length). The RequestFrame call in the diagram is the interface call to the Schedule Table Manager. The LinIf_MainFunction_<ChannelId> gets the frame to send and the delay to the next frame. The AllocateRxBuffer call is to show that the storage of the received frame is made in the LIN Driver and not in the LIN Interface.


BSW Task (OS task or cyclic call)
«module» LinIf User
«module» LinIf
«module» Lin
LinIf_MainFunction_<ChannelId>()
RequestFrame()
Lin_SendFrame(Channel, PduInfoPtr)
AllocateRxBuffer()
LinIf_MainFunction_<ChannelId>()
LinIf_MainFunction_<ChannelId>()
Lin_GetStatus(Channel, Lin_SduPtr) : Lin_StatusType
Lin_GetStatus() :LIN_RX_OK
<User_RxIndication>(RxPduId, PduInfoPtr)
CopyBuffer()

Chapter 3 Tresos Configuration Items

3.1 Container Inclusion Relation

The container inclusion relation is shown below:

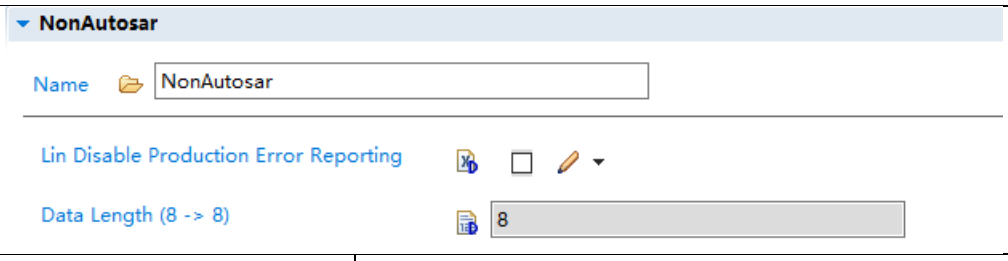


3.2 Containers and Variables

3.2.1 IMPLEMENTATION_CONFIG_VARIANT

Container	IMPLEMENTATION_CONFIG_VARIANT	
Description	N/A	
Screenshot	Configuration Variant VariantPostBuild	
Properties	**Property**	**Value**
	Type	Variable: enumeration
	Symbolic Name	False
	Default	VariantPostBuild
	Range	VariantPostBuild, VariantPreCompile
Returns	N/A	

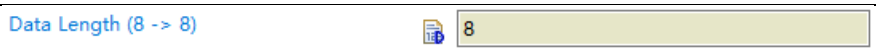
3.2.2 NonAutosar

Container	NonAutosar	
Description	This container contains the parameters if the Non-Autosar.	
Screenshot		
Properties	**Property**	**Value**
	Type	Container

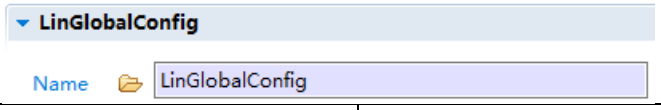
3.2.2.1 LinDisableDemReportErrorStatus

Container	LinDisableDemReportErrorStatus	
Description	Switches the Diagnostic Error Reporting and Notification OFF	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable:boolean
	Symbolic Name	False
	Default	False

3.2.2.2 LinMaxDataLength

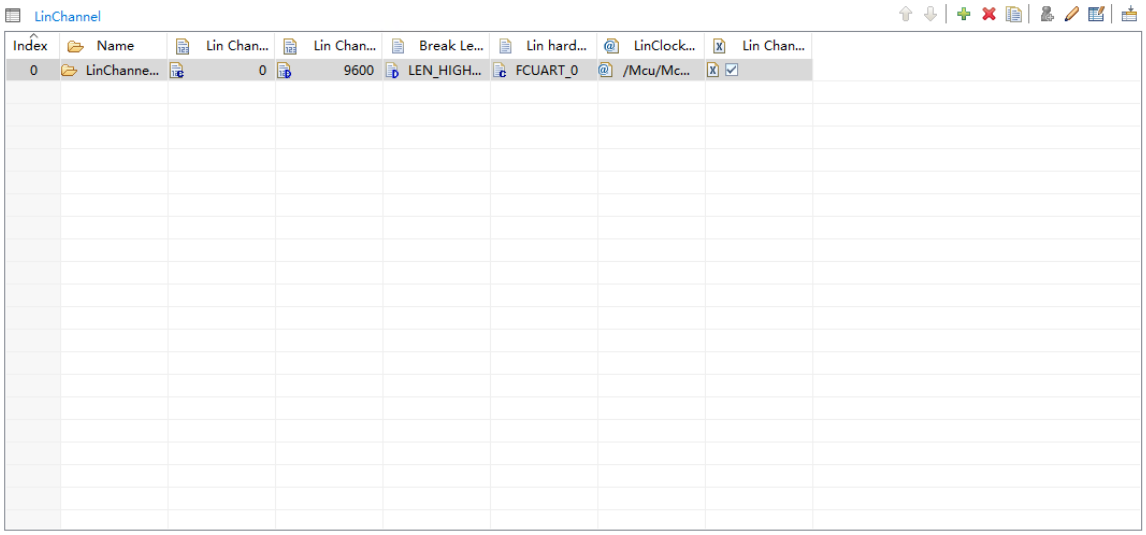
Container	LinMaxDataLength	
Description	Defines the maximum data length of LIN frame	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable:interger
	Symbolic Name	False
	Default	8

3.2.3 LinGlobalConfig

Container	LinGlobalConfig	
Description	This container contains the global configuration parameter of the Lin driver.	
Screenshot		
Properties	**Property**	**Value**
	Type	Container

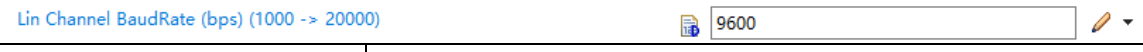
3.2.3.1 LinChannel

List	LinChannel

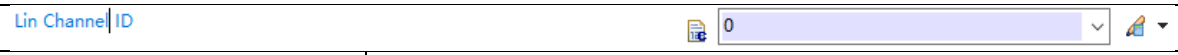
Description	This container contains the configuration (parameters) of the LIN Controller(s)
Screenshot	

Properties	**Property**	**Value**
	Type	Map
	Min	1

3.2.3.1.1 LinChannelBaudRate

Container	LinChannelBaudRate	
Description	Specifies the baud rate of the LIN channel.	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable:integer
	Origin	AUTOSAR_ECUC
	Default	9600
	Range	1000-20000

3.2.3.1.2 LinChannelId

Variable	LinChannelId	
Description	Identifies the LIN channel. Replaces LIN_CHANNEL_INDEX_NAME from the LIN SWS	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable:integer
	Origin	AUTOSAR_ECUC
	Default	0
	Range	0-65535

3.2.3.1.3 BreakLength

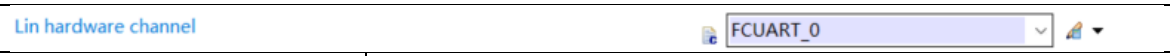
Variable	BreakLength	
Description	Defines the break length in bits. In FCUART break length is fixed to 13bit.	
Screenshot		

Properties	Property	Value
	Type	Variable: Enumeration
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	LEN_HIGHER_13BITS

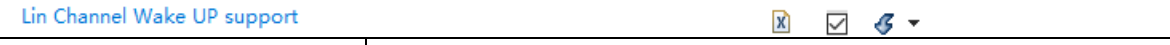
3.2.3.1.4 BreakDelimiter

Variable	BreakDelimiter	
Description	Defines the length delimiter for LIN break send out in bits.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	LEN_DELIMITER_2BITS

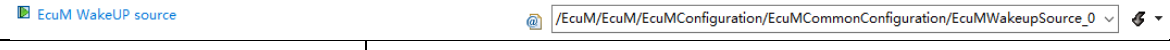
3.2.3.1.5 LinHwChannel

Variable	LinHwChannel	
Description	Selects the Lin Hardware Channel.	
Screenshot		
Properties	Property	Value
	Type	Variable: Enumeration
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	FCUART_0

3.2.3.1.6 LinChannelWakeupSupport

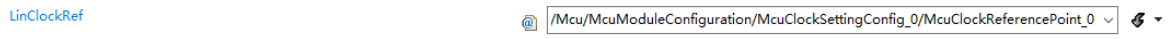
Variable	LinChannelWakeupSupport	
Description	Specifies if the LIN hardware channel supports wake up functionality	
Screenshot		
Properties	Property	Value
	Type	Variable:boolean
	Origin	AUTOSAR_ECUC
	Default	false

3.2.3.1.7 LinChannelEcuMWakeupSource

Variable	LinChannelEcuMWakeupSource	
Description	This parameter contains a reference to the Wakeup Source for this controller as defined in the ECU State Manager.	
Screenshot		
Properties	Property	Value

	Type	Variable:reference
	Origin	AUTOSAR_ECUC

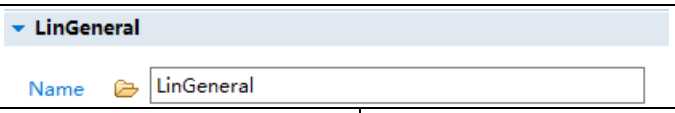
3.2.3.1.8 LinClockRef

Variable	LinClockRef	
Description	This parameter contains a reference to the Wakeup Source for this controller as defined in the ECU State Manager.	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable:reference
	Origin	AUTOSAR_ECUC

3.2.3.1.9 LinChannelEcucPartitionRef

Variabel	LinChannelEcucPartitionRef	
Description	Maps one single Lin channel to zero or one ECUC partitions. The ECUC partition referenced is a subset of the ECUC partitions where the Lin driver is mapped to.	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable:reference
	Origin	AUTOSAR_ECUC

3.2.4 LinGeneral

Container	LinGeneral	
Description	This container contains the parameters related to each LIN Driver Unit.	
Screenshot		
Properties	**Property**	**Value**
	Type	Container

3.2.4.1 LinDevErrorDetect

Variable	LinDevErrorDetect	
Description	Switches the development error detection and notification on or off. • true: detection and notification is enabled. • false: detection and notification is disabled	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable: Integer
	Origin	AUTOSAR_ECUC
	SymbolicNameValue	False
	Default	False

3.2.4.2 LinMulticoreSupport

Variable	LinMulticoreSupport	
Description	This parameter determine multi-core feature will be used in Lin driver. • true: LinMulticoreSupport is enabled. • false: LinMulticoreSupport is disabled	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable: Integer
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	False

3.2.4.3 LinIndex

Variable	LinIndex	
Description	Specifies the InstanceId of this module instance. If only one instance is present it shall have the Id 0	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable: Integer
	Origin	AUTOSAR_ECUC
	SymbolicNameValue	False
	Range	0-255

3.2.4.4 LinTimeoutDuration

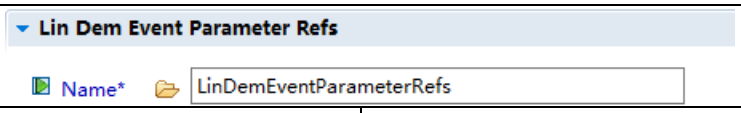
Variable	LinTimeoutDuration	
Description	Specifies the maximum number of loops for blocking function until a timeout is raised in short term wait loops	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable: Integer
	Origin	AUTOSAR_ECUC
	SymbolicNameValue	False
	Range	0-65535

3.2.4.5 LinVersionInfoApi

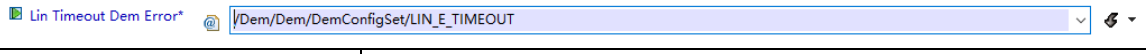
Variable	LinVersionInfoApi	
Description	Switches the Lin_GetVersionInfo function ON or OFF.	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable:Boolean
	Origin	AUTOSAR_ECUC
	SymbolicNameValue	False

	Default	False

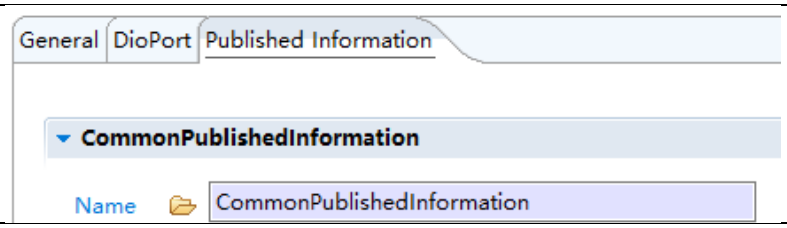
3.2.5 LinDemEventParameterRefs

Variable	LinDemEventParameterRefs	
Description	Container for the references to DemEventParameter elements which shall be invoked using the API Dem_SetEventStatus in case the corresponding error occurs. The EventId is taken from the referenced DemEventParameter's DemEventId symbolic value. The standardized errors are provided in this container and can be extended by vendorspecific error references.	
Screenshot		
Properties	**Property**	**Value**
	Type	Contianer

3.2.5.1 LIN_E_TIMEOUT

Variable	LIN_E_TIMEOUT	
Description	Reference to the DemEventParameter which shall be issued when the error "Timeout caused by hardware error" has occurred. If the reference is not configured the error shall be reported as DET error.	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable: reference
	Origin	AUTOSAR_ECUC

3.2.6 CommonPublishedInformation

Container	CommonPublishedInformation	
Description	Common container, aggregated by all modules. It contains published information about vendor and versions.	
Screenshot		
Properties	N/A	

3.2.6.1 ArReleaseMajorVersion

Variable	ArReleaseMajorVersion	
Description	Major version number of AUTOSAR specification on which the appropriate implementation is based.	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False

	Default	4

3.2.6.2 ArReleaseMinorVersion

Variable	ArReleaseMinorVersion	
Description	Minor version number of AUTOSAR specification on which the appropriate implementation is based.	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	6

3.2.6.3 ArReleaseRevisionVersion

Variable	ArReleaseRevisionVersion	
Description	Revision version number of AUTOSAR specification on which the appropriate implementation is based.	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	0

3.2.6.4 SwMajorVersion

Variable	SwMajorVersion	
Description	Major version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	0

3.2.6.5 SwMinorVersion

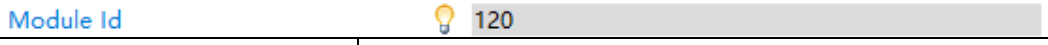
Variable	SwMinorVersion	
Description	Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable: Integer_Label
	Origin	FLAGCHIP

	SymbolicNameValue	False
	Default	3

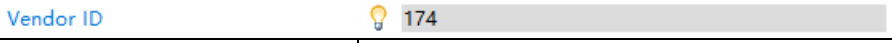
3.2.6.6 SwPatchVersion

Variable	SwPatchVersion	
Description	Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	0

3.2.6.7 ModuleId

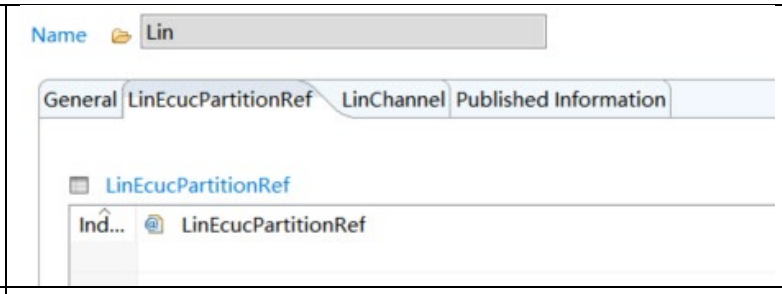
Variable	ModuleId	
Description	Module ID of this module from Module List.	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	120

3.2.6.8 VendorId

Variable	VendorId	
Description	Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.	
Screenshot		
Properties	**Property**	**Value**
	Type	Variable: Integer_Label
	Origin	FLAGCHIP
	SymbolicNameValue	False
	Default	174

3.2.7 LinEcucPartitionRef

Container	CommonPublishedInformation
Description	Maps the Lin driver to zero or multiple ECUC partitions to make the modules API available in this partition. The Lin driver will operate as an independent instance in each of the partitions.

Screenshot	
Properties	N/A

Chapter 4 Configuration Guides

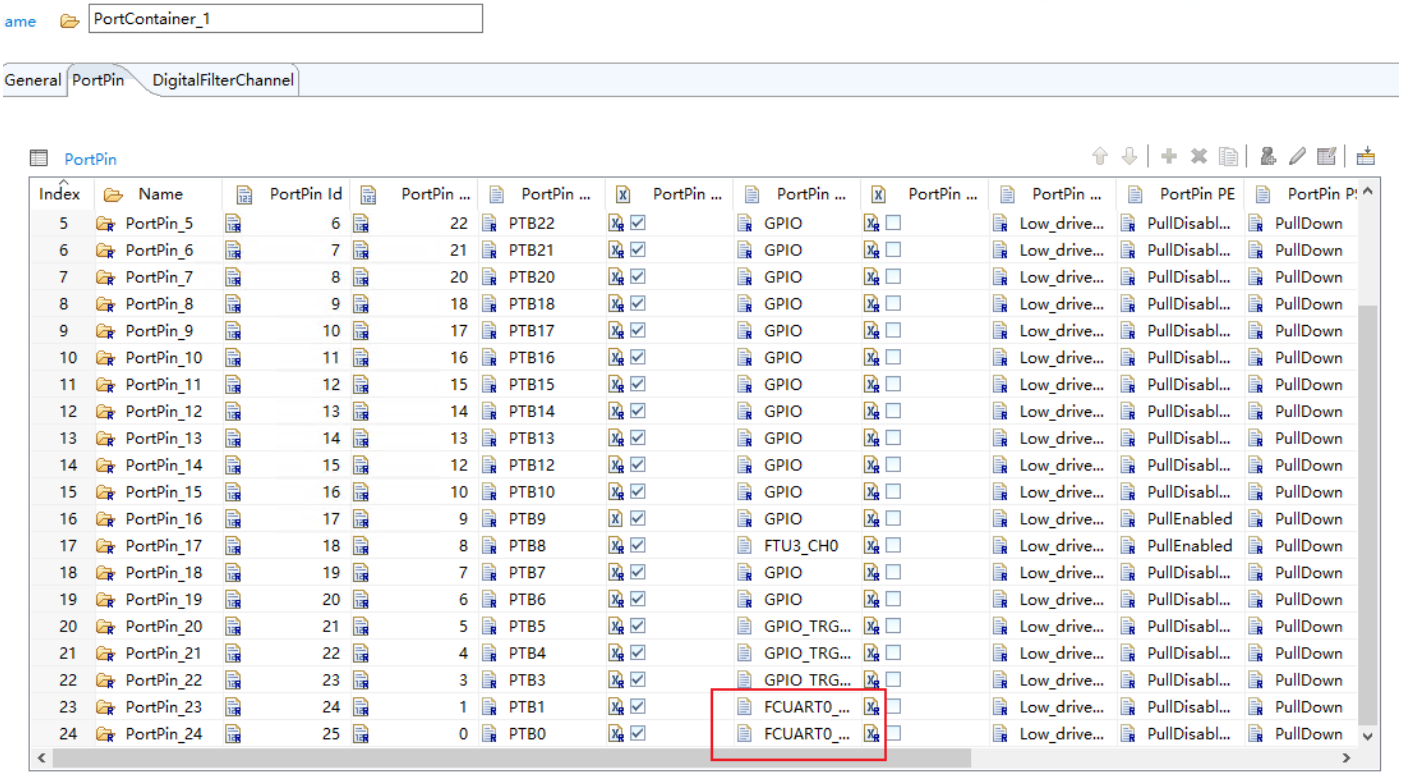
4.1 LIN Usage Common Steps

Basically, the LIN module can be configured by following the below 3 steps:

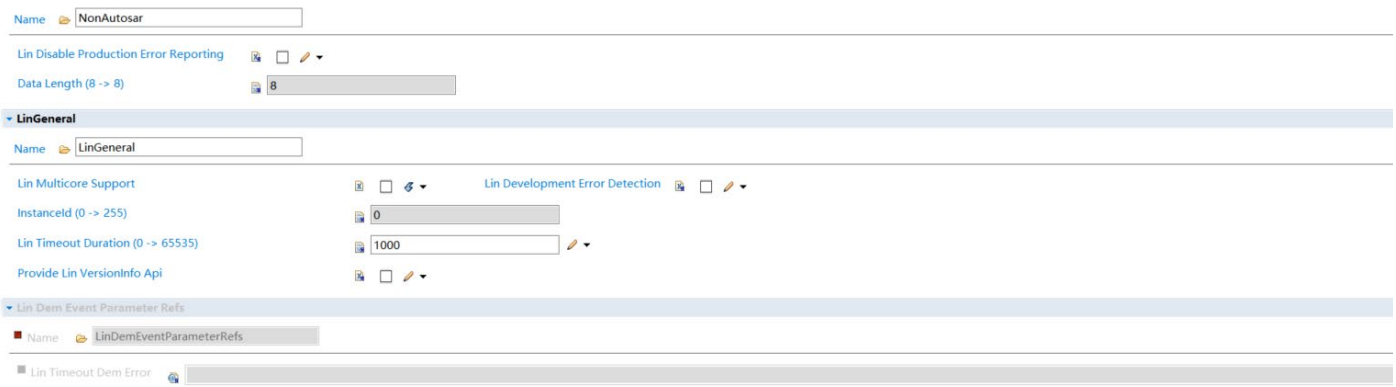
1) Configure LIN general configurations and enable UART in Port configuration.
2) Configure LIN channel configurations and select wakeup support if needed.
3) Generate configuration files.

4.2 LIN Channel Demo

1) Set the port pin PTB0 and PTB1 to UART. (Use FCUART0 for example)



2) Set general configuration. Dem is enabled if Lin Disable Production Error Reporting is unchecked. Tick the Lin Development Error Detection box if Det is needed.



3) Set Lin channel baud rate and specify the UART number. In this demo, FCUART0 is selected, and the baud rate is 9600. Choose the clock reference FCUART0. Wakeup support and wakeup source can only be set after the source reference is created in EcuM.